

Complemento Práctico: El Poder del Polimorfismo Paramétrico

Generics, Wildcards y la Flexibilidad Total en Java

¿Alguna vez te preguntaste por qué ArrayList funciona para guardar String, Integer, Perro o cualquier objeto sin que tengas que descargar una librería distinta para cada uno? La respuesta es **Generics** (Genéricos). En esta unidad, descubriremos cómo Java nos permite escribir código que se adapta al tipo de dato que le lancemos.



Contenido

El Concepto

¿Qué es <T> y por qué lo amamos?

Tu propia Clase Genérica

Creando el Sorteador<T>

El Problema de la Invarianza

Por qué List<String> NO es List<Object>

Wildcards

El comodín ? y sus restricciones

Bounded Wildcards

La diferencia vital entre ? extends y ? super

Espiando al ArrayList: El Nacimiento de los Genéricos

Antes de Java 5, las colecciones trabajaban con `Object`. Esto obligaba a hacer *casting* todo el tiempo y era inseguro: podías meter un Perro en una lista de Gatos y el error saltaba recién al ejecutar. Imagina el caos en una aplicación grande donde un error de tipos podría permanecer oculto hasta que un usuario lo encontrara en producción.

Con los **Genéricos**, aplicamos el "Polimorfismo Paramétrico": pasamos el tipo de dato como si fuera un parámetro más. Es como decirle a una clase: "Prepárate para trabajar con cualquier tipo, pero una vez que te diga cuál, comprométete con ese tipo específico".

La Sintaxis del Diamante < >

Cuando escribes:

```
ArrayList<String> lista = new ArrayList<>();
```

Le estás diciendo a la clase `ArrayList`: *"Oye, en esta instancia, tu tipo genérico E será String"*.

Automáticamente, todos los métodos de esa lista (`add`, `get`) se transforman para aceptar y devolver solo `String`.

Beneficios Clave



Seguridad de Tipos

El compilador te avisa si intentas meter un dato incorrecto



Código Limpio

Te olvidas de hacer `(String) obj` cada vez que sacas un dato

Creando Tu Propia Clase Genérica: El Sorteador

En el video creamos una clase llamada **Sorteador**. Su misión es guardar elementos y devolver uno al azar (sortearlo). ¿De qué tipo? ¡De cualquiera! Por eso usamos `<T>`. Esta pequeña letra T representa un "Type" (tipo) que será decidido cuando alguien cree una instancia de nuestra clase.

Código Maestro

```
// <T> indica que esta clase recibe un "Tipo" genérico
public class Sorteador<T> {
    private ArrayList<T> lista; // Usamos T internamente

    public Sorteador() {
        this.lista = new ArrayList<>();
    }

    // El método recibe un T (sea lo que sea T)
    public void agregar(T elemento) {
        this.lista.add(elemento);
    }

    public T sortear() {
        if (lista.isEmpty()) return null;
        int indice = new Random().nextInt(lista.size());
        return lista.get(indice); // Devuelve un T
    }
}
```

Uso con Strings

```
Sorteador<String> sorteoNombres = new
Sorteador<>();
sorteoNombres.agregar("Ana");
sorteoNombres.agregar("Juan");
String ganador = sorteoNombres.sortear();
```

Uso con Enteros

```
Sorteador<Integer> sorteoLoteria = new
Sorteador<>();
sorteoLoteria.agregar(42);
sorteoLoteria.agregar(17);
Integer numero = sorteoLoteria.sortear();
```

Observa la magia: escribimos la clase **una sola vez**, y funciona perfectamente para Strings, Integers, objetos Perro, o cualquier cosa que se te ocurra. Esto es reutilización de código en su máxima expresión.

El Comodín (?) y el Problema de la Invarianza

Aquí empieza la complejidad real. Presta atención a esta regla de oro que sorprende a muchos desarrolladores noveles:

❏ **ArrayList<String> NO es hijo de ArrayList<Object>.**

Si tienes un método que acepta ArrayList<Object>, **no puedes** pasarle un ArrayList<String>. Da error de compilación. ¿Por qué esta restricción aparentemente extraña? Porque si Java te dejara, dentro del método podrías hacer `lista.add(new Integer(5))` y corromperías la lista de Strings original. Estarías violando la promesa de que esa lista solo contiene Strings.

El Problema

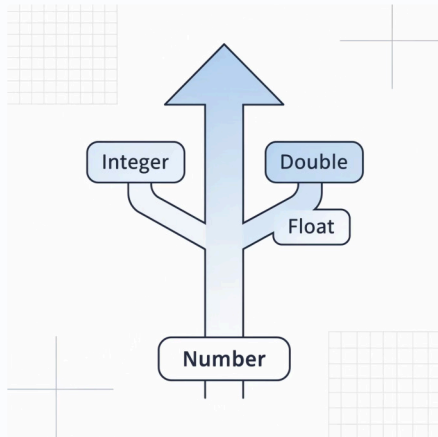
```
public void
imprimir(ArrayList<Object> lista) {
    // No puedes pasar
    ArrayList<String> aquí
    ...
}
```

La Solución: El Wildcard (?)

```
public void imprimir(ArrayList<?>
lista) {
    for (Object o : lista) {
        System.out.println(o);
    }
}
```

El Precio a Pagar: Al usar <?>, la lista se vuelve de **Solo Lectura** (lógicamente). No puedes agregar nada (salvo null), porque Java no sabe de qué tipo es la lista realmente. Es un compromiso inteligente: ganas flexibilidad de entrada a cambio de restricciones de modificación.

Comodines Limitados: Upper Bounded (? extends)



A veces el comodín simple (?) es demasiado amplio. Queremos decir: *"Cualquier lista, siempre que sea de Números (o sus hijos)"*. Esto se llama **Límite Superior** o Upper Bounded Wildcard.

Sintaxis

```
List<? extends Number>
```

Esta declaración acepta List<Integer>, List<Double>, List<Float>, o cualquier otra lista de tipos que hereden de Number. Es perfecta para operaciones de **lectura y cálculo** donde necesitas garantizar que todos los elementos comparten ciertas capacidades (como convertirse a double).

Caso de Uso Real: Sumar Valores

```
public double sumar(List<? extends Number> lista) {
    double suma = 0;
    for (Number n : lista) { // Seguro puedo tratarlo como Number
        suma += n.doubleValue();
    }
    return suma;
}
```

Ventaja

Java garantiza que lo que salgas será un Number, permitiéndote usar métodos como doubleValue(), intValue(), etc.

Restricción

Al igual que con ?, **NO puedes agregar elementos**. Solo leer. Java no sabe qué tipo específico necesita para entrar (¿Integer? ¿Double?), así que prohíbe la adición para mantener la seguridad.

Comodines Limitados: Lower Bounded (? super)

Este es el menos intuitivo pero vital para operaciones de escritura. Queremos decir: *"Cualquier lista, siempre que sea de Futbolistas (o sus padres, como Deportista o Object)"*. Esto se llama **Límite Inferior** o Lower Bounded Wildcard.

Sintaxis

```
List<? super Futbolista>
```

Esta declaración acepta List<Futbolista>, List<Deportista>, List<Object>, o cualquier lista de tipos que sean superclases de Futbolista. Es perfecta para operaciones donde necesitas **agregar** elementos.

Caso de Uso Real: Llenar una Lista

```
public void agregarSuplentes(List<? super Futbolista> lista) {
    // Puedo agregar Futbolistas porque sé que la lista es
    // de Futbolistas, o de Deportistas, o de Objetos.
    // En cualquiera de esos casos, un Futbolista "entra" bien.
    lista.add(new Futbolista("Messi"));
    lista.add(new Futbolista("Cristiano"));
}
```

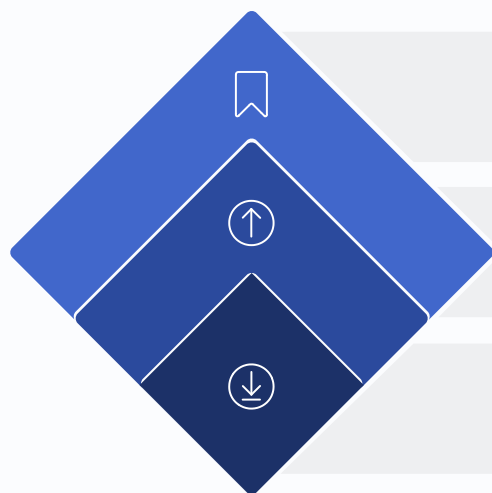
¿Por qué funciona?

Si la lista es de Deportistas, un Futbolista cabe perfectamente (es un Deportista). Si es de Objects, también cabe (todo es Object). El principio de sustitución de Liskov garantiza que puedes poner un hijo donde se espera un padre.

¿Por qué no extends?

Si usaras <? extends Futbolista>, la lista podría ser de Arquero (subclase de Futbolista). ¿Puedes meter un Delantero en una lista de Arqueros? No. Por eso extends bloquea la escritura.

La Regla PECS: Tu Mejor Aliada



Regla PECS

Recuerda Producer Extends / Consumer Super

Consumidor: Super

Escritura y adición segura

Productor: Extends

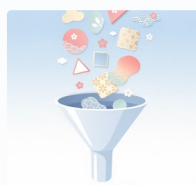
Lectura segura de datos

Para recordar cuándo usar cada wildcard, los expertos en Java usan la regla mnemotécnica **PECS**, que significa: **Producer Extends, Consumer Super**.



Producer Extends

Si la colección **produce** datos (tú los sacas, los lees, los procesas), usa `<? extends T>`. Ejemplo: método que suma números, método que imprime elementos, método que encuentra el máximo.



Consumer Super

Si la colección **consume** datos (tú le agregas, la llenas, la modificas), usa `<? super T>`. Ejemplo: método que agrega elementos, método que copia datos hacia la lista, método que inserta registros.



Truco mental: Extends mira "hacia abajo" en la jerarquía (hacia clases más específicas) y te deja leer. Super mira "hacia arriba" (hacia clases más generales) y te deja escribir.

Comparación Visual: Tres Sabores de Wildcards



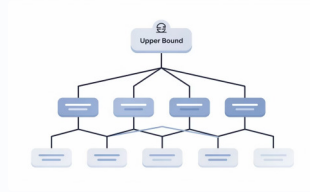
Comodín Simple (?)

Sintaxis: `List<?>`

Significa: "Cualquier tipo desconocido"

Uso: Cuando solo necesitas iterar o verificar tamaño

Operaciones: Solo lectura como Object



Límite Superior (? extends T)

Sintaxis: `List<? extends Number>`

Significa: "T o cualquier subclase de T"

Uso: Cuando necesitas leer/procesar datos

Operaciones: Puedes obtener como T, no puedes agregar



Límite Inferior (? super T)

Sintaxis: `List<? super Futbolista>`

Significa: "T o cualquier superclase de T"

Uso: Cuando necesitas agregar/escribir datos

Operaciones: Puedes agregar T, lees como Object

Resumen: Abriendo la Caja de Pandora

Hemos abierto la caja de Pandora de los tipos en Java. Lo que comenzó como una simple pregunta sobre cómo ArrayList maneja diferentes tipos nos llevó a un viaje profundo por uno de los sistemas de tipos más sofisticados en programación orientada a objetos.

Clases Genéricas

Usamos `<T>` para definir plantillas reutilizables que funcionan con cualquier tipo

Invarianza

Entendimos que las colecciones no respetan la herencia tradicional para proteger la memoria

Wildcards

- `<?>`: "Cualquier cosa" (Solo lectura)
- `<? extends T>`: "T o inferiores" (Ideal para leer)
- `<? super T>`: "T o superiores" (Ideal para escribir)

Conceptos Clave Dominados

- Polimorfismo paramétrico vs. polimorfismo tradicional
- Seguridad de tipos en tiempo de compilación
- La regla PECS para recordar cuándo usar cada wildcard
- Por qué la invarianza protege tu código de errores sutiles

Próximos Pasos

En la próxima actividad, pondremos toda esta teoría en práctica. Usaremos genéricos para ordenar listas de objetos complejos y crear buscadores inteligentes que funcionen con cualquier tipo de dato. ¡El poder estará en tus manos!